Week 07 Candidate Move Generation

Project: Research Design and Implementation of a Gomoku Game Engine with Artificial Intelligence
Reporting period: Week 07
Report date: 17 October 2026
Status: Implemented in the isolated Rust reference core and evaluated with local experiments
Evidence scope: Native reference implementation and reproducible research fixtures

1 Abstract

Candidate generation reduces the moves presented to a game tree search. This week examines the spatial policy actually implemented in caro-core-reference, rather than assuming that every legal move is searched. The policy includes empty cells within a configurable Chebyshev radius of any occupied cell. Its default radius is two and its default candidate limit is absent. An empty board receives the center move. These choices reduce branching but introduce a different decision problem from unrestricted minimax.

The contribution is a precise definition, correctness arguments for geometric membership, executable geometry and search comparisons, and an explicit boundary between immediate tactical membership and general strategic completeness. A candidate cap is especially significant: the implementation truncates coordinates in row major order before search or move ordering can score them. Therefore a cap can remove an immediate winning move. The report does not claim a universally optimal radius, a complete tactical engine, or integration with the browser application.

2 Research Questions and Evidence Convention

RQ 07 1 asks how radius changes candidate cardinality for controlled spatial configurations. RQ 07 2 asks which membership and determinism properties follow from the implementation. RQ 07 3 asks what can be established about immediate winning moves and what a hard cap invalidates. RQ 07 4 asks whether smaller candidate trees change the searched value or measured work on the selected fixtures.

Throughout this report, [A] denotes inspected source or newly recorded local execution, [B] denotes a primary publication or official rule source, [C] denotes a derivation or inference with stated assumptions, and [D] denotes a recommendation. Observing source in the reference crate is evidence about that crate. It does not establish that the same implementation is compiled into a deployed Wasm binary.

3 Source Context and Responsibility

File	Responsibility	Observed evidence and confidence
caro-core-reference/ src/model.rs	Coordinates and board size	[A] BOARD_SIZE is 15 and Move::as_index uses row times 15 plus column. High confidence.
caro-core-reference/ src/movegen.rs	Candidate membership and sequence	[A] generate_candidates marks empty neighbors, emits increasing indices, then applies truncate. Default radius is 2 and default cap is None. High confidence.
caro-core-reference/src/ state.rs	Legal moves and state transitions	[A] GameState::legal_moves returns empty cells only while status is Playing; play and undo own temporary transitions. High confidence.

File	Responsibility	Observed evidence and confidence
caro-core-reference/src/search/mod.rs	Search configuration and policy selection	[A] generate_moves dispatches between AllLegal and Candidates. High confidence.
caro-core-reference/src/search/alpha_beta.rs	Search traversal	[A] Candidate generation occurs before optional ordering and recursive expansion. High confidence.

The flow is state occupancy to geometric marking to ordered coordinate list to optional truncation to search. Candidate generation owns membership. The evaluator owns scores. Alpha beta owns bound based traversal. Keeping these responsibilities distinct makes it possible to identify whether a changed result came from a changed search space or from a search defect.

4 Mathematical Definition

Let the board domain be

$$\Omega = \{0, \ldots, 14\} \times \{0, \ldots, 14\}.$$

For a playing state s, let $O(s)$ be the occupied cells and $A(s) = \Omega \setminus O(s)$ be all legal empty cells. Coordinates in this report are zero based and written as row then column. Define the Chebyshev distance

$$d_\infty((r, c), (u, v)) = \max(|r - u|, |c - v|).$$

For nonempty occupancy and integer radius $R \geq 0$, the uncapped candidate set is

$$C_R(s) = \{m \in A(s) : \exists o \in O(s), d_\infty(m, o) \leq R\}.$$

For an empty board the implementation overrides that expression and returns $\{(7, 7)\}$. Let $\mathrm{row}(C)$ mean increasing index order. With cap K, the returned sequence is the first $\min(K, |C_R|)$ elements of $\mathrm{row}(C_R)$. The empty board return happens before this cap operation, so even cap zero returns the center on an empty board. This is a source behavior, not a mathematical necessity.

Candidate restriction changes the action set. Writing $V_A(s, d)$ for depth limited minimax over all legal moves and $V_C(s, d)$ for minimax over recursively generated candidates, no general equality follows. For a maximizing node whose descendant values were held fixed, taking a maximum over a subset cannot increase that maximum. In a real alternating tree the restrictions affect both players and descendants, so a simple one direction inequality for the final root values should not be assumed.

5 Membership Properties and Proofs

[C] Soundness holds for playing states: every marked coordinate passes the board boundary check and the emptiness check, so every returned candidate is legal. Duplicate neighborhoods cannot duplicate a move because the marking array stores one Boolean per board cell. The final scan visits each index once, so the resulting sequence is deterministic and contains no duplicates.

[C] Uncapped monotonicity follows directly from the distance inequality. If $R_1 \leq R_2$, then $C_{R_1}(s) \subseteq C_{R_2}(s)$. This is a set statement. Under caps, increasing radius can insert earlier coordinates into the retained prefix and displace a previously retained move, so retained prefix membership need not be monotone.

[C] Nonemptiness requires assumptions. On a nonempty, nonfull board, with radius at least one and no zero cap, there is an occupied cell adjacent to an empty cell. To see this, take a path through the connected board grid from any occupied cell to any empty cell; the first transition crosses that

boundary. The empty endpoint is a candidate. Radius zero on a nonempty board returns no candidates. A zero cap also removes every candidate on a nonempty board. Therefore the unconditional assertion that the generator returns moves whenever legal moves exist is false.

The generator itself does not check terminal status. Calling it directly on a finished board can return empty cells even though legal_moves returns none. Search avoids this mismatch by checking terminal status before generation. Soundness as a game action contract consequently includes the playing state precondition.

6 Immediate Tactical Membership

[B] Freestyle Gomoku accepts a continuous line of five or more stones; official Gomocup information distinguishes that rule from exact five and Standard Caro. This report uses freestyle fixtures. Gomocup rule definitions.

[C] A narrow tactical property can be proved without declaring all tactics complete. Suppose a legal move m completes a continuous line of at least five for player p. Within a five cell segment containing m, the other cells already contain p. At least one cell immediately adjacent to m along the winning direction is therefore occupied. Its Chebyshev distance from m is one. Hence every such completion lies in the uncapped $C_R(s)$ when $R \geq 1$.

The same argument applies to an opponent's one move winning completion considered as a blocking coordinate. This proves that the coordinate is represented. It does not prove that choosing that coordinate prevents every loss, that the evaluator recognizes the threat, or that search explores enough depth. Two independent opponent winning completions may make a loss unavoidable. Longer combinations, remote development moves, and variant forbidden move rules need separate reasoning.

The previous claim that a gap in a pattern automatically escapes radius one was incorrect. In XX.XX, the gap is adjacent to stones on both sides. A radius one spatial generator includes it even when a contiguous run evaluator fails to assign the intended threat score. Candidate membership and pattern recognition are separate questions.

7 Algorithm and Complexity

The following pseudocode preserves the implemented sequence and its empty board exception.

```
GenerateCandidates state radius cap
  if move count is zero return center
  marked = false for every board cell
  for each occupied cell o
    for each offset in square of radius
      m = o plus offset
      if m is in bounds and empty mark m
  moves = marked coordinates in row major order
  if cap exists truncate moves to cap
  return moves
```

[C] Let N be board width and k the occupied count. Scanning occupancy and the marking array costs $O(N^2)$. Neighborhood visits cost $O(k(2R+1)^2)$, giving total time $O(N^2 + kR^2)$. The Boolean marking array and output require $O(N^2)$ memory. Radius and board size are bounded in the present program, but retaining parameters clarifies scaling. Truncation reduces recursive branching; it does not avoid the preceding neighborhood marking work.

For a single interior stone with its whole radius square inside the board, the exact count is $(2R+1)^2 - 1$. Thus radii one, two, and three yield 8, 24, and 48 candidates. With one stone in the corner, the clipped count is $(R+1)^2 - 1$, yielding 3, 8, and 15. These are geometric derivations, not estimates of typical game phases.

8 Worked Examples

Place X at $(7,7)$. There are 224 legal moves. Radius two returns 24 cells, so the reduction is

$$1-\frac{24}{224}=0.892857\approx 89.29\,\%.$$

The old value of eight radius two candidates for this position described radius one and did not match the current source. This example also demonstrates why a reported count must identify actual coordinates rather than only a label such as opening.

For the cap counterexample, place X at $(7,3),(7,4),(7,5),(7,6)$, O at $(5,5),(5,6)$, and select X to move. Winning completions include $(7,2)$ and $(7,7)$. Radius one includes both. Its first six coordinates in row major order are around the earlier O stones, however. Applying cap six removes both winning completions. Ordering the six survivors afterward cannot recover them. This is an explicit mechanism for tactical loss under a cap, not an argument that a cap always loses.

9 Experimental Method

The new native harness is stored under research/week_05_09. It calls the existing crate through a separate research package and records machine readable evidence. Geometry fixtures vary position while holding occupancy semantics and coordinate conventions fixed. Search comparisons use identical depth, weights, rules, and ordering settings, changing only the move policy. Search fixtures are constructed with the crate's fixture helpers; they are controlled states rather than claims about a recorded human game.

Candidate count is a root property; candidate_moves_generated is a sum over expanded internal nodes. They are not interchangeable. Likewise, node reduction measures the declared search tree and cannot alone establish better play. Search duration is local native execution on the recorded host and includes the evaluation cost of that implementation.

10 Experimental Results

[A] The frozen evidence contains 48 geometry configurations, all passing emptiness, uniqueness, and radius checks. The uncapped counts below come from results/candidates.csv; radius zero returns zero for each nonempty fixture. Clustered occupies the four cells of the square from $(7,7)$ to $(8,8)$; separated occupies $(2,2)$ and $(12,12)$.

Fixture	Legal	Radius 1	Radius 2	Radius 3	Radius 2 reduction
Empty	225	1	1	1	99.56 percent
Center	224	8	24	48	89.29 percent
Corner	224	3	8	15	96.43 percent
Clustered	221	12	32	60	85.52 percent
Separated	223	16	48	70	78.48 percent
Win	219	20	46	80	79.00 percent

[A] At depth two with ordering disabled, all legal and uncapped radius one alpha beta returned the same values on the following controlled fixtures. Durations are medians of three search calls in microseconds.

Fixture	Full nodes	Radius 1 nodes	Shared score	Full median	Radius 1 median
Center	21986	36	−100	16954	23
Clustered	10939	64	0	16442	41
Win	11922	237	9999999	9035	167

[A] results/cap_loss.json independently records that cap six reduces the win fixture from 20 to six candidates, omits both endpoints, and changes depth one score from 9999999 to 99990. This counterexample directly rejects tactical safety of arbitrary row major truncation.

Reproduce with python3 research/week_05_09/run.py. results/provenance.json records Rust 1.98.1, release mode, Linux x86 64, and an Intel Xeon W 11855M host. Timing excludes state cloning and TT or hash initialization; host load was uncontrolled. Full raw trials, minima, maxima, source hashes, and verification counts remain alongside the report. These measurements support fixture specific work reduction, not general value equality or a universal speedup.

11 Interpretation and Limitations

[C] Spatial filtering is useful because a compact neighborhood can replace hundreds of empty cells with tens of candidates. That observation supports an engineering tradeoff, not a theorem that excluded cells are irrelevant. The empty board center policy uses symmetry as a practical convention and still changes the action set from 225 choices to one.

No universal best radius follows from a small geometry corpus. Coverage depends on stone placement, board edges, clustering, and the tactical horizon. A radius two union around several separated clusters can be much larger than the union around a compact cluster with the same stone count. Candidate reduction should therefore be reported per fixture and later evaluated on representative positions.

The current evaluator counts maximal contiguous runs. Broken threats require additional pattern logic before any claim of complete tactical evaluation. Neither native counts nor a proof about immediate completions establishes playing strength, Wasm performance, browser responsiveness, or compatibility with a deployed game. Full legal and candidate searches may correctly return different values because they solve different trees.

12 Traceability and Next Steps

Question	Evidence	Bounded conclusion
RQ 07 1	movegen.rs and geometry evidence	Radius and clipping determine measured root counts.
RQ 07 2	Marking algorithm and parameter counterexamples	Legal membership and determinism hold under stated preconditions.
RQ 07 3	Adjacency proof and cap fixture	Uncapped immediate completions are included; arbitrary caps do not preserve them.
RQ 07 4	Native policy comparison	Work and values describe only the specified fixtures and depths.

[D] Keep the default uncapped policy while investigating radius selection. If a budgeted shortlist is later required, first include explicitly detected immediate wins and necessary defense coordinates, then

score remaining moves before capping. Such a policy needs its own safety tests and evidence because it would change membership ownership. [D] Add broader game positions before selecting a permanent radius and evaluate tactical accuracy separately from speed. This week delivers a reproducible reference study; product adoption remains a separate integration task.

13 References

1. [A] caro-core-reference/src/movegen.rs, generate_candidates and CandidateConfig, inspected 17 October 2026.
2. [A] caro-core-reference/src/state.rs, GameState::legal_moves, play, and undo, inspected 17 October 2026.
3. [A] caro-core-reference/src/search/mod.rs and src/search/alpha_beta.rs, policy dispatch and traversal, inspected 17 October 2026.
4. [A] research/week_05_09, local harness and recorded evidence accompanying this report.
5. [B] Gomocup. Detail Information and Gomoku Rules, accessed 17 October 2026.